COL380 Assignment 2 – Report

Popat Nihal Alkesh (2023CS10058)

1 Cross-Cutting Design Decisions

Several design choices are shared across all three approaches:

Structure-of-Arrays (SoA) layout. Point data is stored as four separate `int` arrays (`x[], y[], z[], I[]`) rather than as an array of structs. This way, when a warp of 32 threads accesses `x[i]` through `x[i+31]`, the addresses are contiguous in memory, achieving fully coalesced 128-byte memory transactions on the GPU.

Batched memory allocations. Every allocation path performs a *single* `cudaMalloc` (or `new`) and then carves out sub-regions via pointer arithmetic. On the host side, one `new int[5n]` backs all five arrays (`x, y, z, I, updated_ints`). On the device side, one `cudaMalloc` of `5n` ints backs `d_x` through `d_updated_ints`, and all four input arrays are transferred to the GPU in a *single* `cudaMemcpy` call covering `4n` contiguous ints. Similarly, K-Means packs centroids, labels, and the changed-flag into one device buffer (`d_km_buf`), and histogram + CDF into one host buffer (`h_hist_buf`). IVF packs its index arrays into two device buffers as well. This eliminates repeated allocator overhead, since each `cudaMalloc` call can cost 0.1–1 ms due to driver synchronization.

Priority queue with split key/value arrays. The max-heap priority queue stores distances in a `key[]` array and point indices in a separate `val[]` array, both stack-allocated in per-thread local memory. Because the compiler can place small fixed-size arrays in registers or L1-cached local memory, the distance comparison path (which only touches `key[]`) benefits from register-level latency without having to load index data on every comparison. All PQ functions use `__forceinline__` to eliminate function-call overhead in the inner loop.

`int32` squared distances. With coordinates in $[-10^4,\ 10^4]$, the maximum squared Euclidean distance is $3 \times (2 \times 10^4)^2 = 1.2 \times 10^9$, which fits comfortably within `INT_MAX` ($\approx 2.1 \times 10^9$). This lets us use 32-bit integer arithmetic throughout, avoiding the cost of 64-bit operations (which have roughly half the throughput on most GPU architectures).

2 Exact KNN

The entire pipeline (neighbour search, histogram construction, CDF computation, and intensity remapping) runs inside a **single fused CUDA kernel**. This avoids storing intermediate neighbour lists in global memory: once a thread finishes its KNN search, it immediately builds the histogram from its PQ contents (still in local memory) and writes only the final equalized intensity back to global memory.

Tiled shared-memory reuse. Each block of `TILE = 128` threads is responsible for 128 query points. The n candidate points are processed in tiles of 128: each tile is cooperatively loaded into four shared-memory arrays (`y_tile_x/y/z/I`, totalling 2 KB), and then *every* thread in the block reads from this shared copy to compute distances against its own query point. This converts what would be 128×128 global memory reads per tile into just 128 global loads followed by 128×128 fast shared-memory reads, reducing global memory traffic by a factor of 128.

Sentinel-based masking. Threads with index $i \geq n$ assign themselves a sentinel intensity of 256 (an impossible value). They continue to participate in shared-memory loads and `__syncthreads()` barriers (which is necessary to prevent deadlocks), but they skip all distance computation and heap work thanks to the guard condition `if(pt_i_I != 256)`. This keeps the control flow uniform within each warp, minimizing divergence.

Per-point histogram slices. Each thread writes to its own disjoint slice of the histogram array at `histogram[i*256 ...i*256+255]`, which is pre-zeroed via a single `cudaMemset` call before the kernel launch. Because the slices do not overlap, no atomic operations are needed.

Complexity: $O(n \log k)$ work per thread, $O(n^2 \log k)$ total.

3 K-Means Clustering

This approach uses a hybrid GPU/CPU loop: cluster *assignment* runs on the GPU, while centroid *recomputation* runs on the CPU with OpenMP.

Shared-memory centroid caching. The `assign_clusters` kernel loads all k centroids ($3k$ integers) into dynamic shared memory once per block. After that, every thread in the block (up to 256 threads) computes its distances to all k centroids entirely from shared memory. This turns what would be $256 \times k$ global reads into just $3k$ global loads per block, giving a $256\times$ reduction for the centroid reads.

Non-atomic convergence flag. A single device-side integer `d_changed` is set to 1 by any thread whose cluster label changes. Since every writing thread stores the same value, this is safe without using `atomicOr`, saving one atomic operation per point per iteration.

OpenMP centroid update with thread-local accumulation. Each OpenMP thread maintains its own private arrays for partial coordinate sums and point counts, which are then merged using an `omp critical` section. This avoids the need for GPU-side atomic reductions and also eliminates false sharing between CPU cores.

CPU-side histogram equalization. After convergence, we observe that we only need *one histogram per cluster*, not one per point. This is because the neighbourhood $N_k(i)$ in K-Means is defined as the entire cluster, so every point in the same cluster shares the same histogram. These per-cluster histograms and CDFs are computed using two parallelized OpenMP loops (one over the k clusters, one over the n points for remapping). The memory footprint is only $O(256k)$, which is far smaller than the $O(256n)$ required by the per-point approaches.

Complexity: $O(k)$ work per thread per iteration, $O(Tnk)$ total; $O(256k + n)$ for equalization.

4 Approximate KNN via IVF

The idea behind the Inverted File Index (IVF) approach is to first partition the point cloud into spatial clusters using K-Means, and then, for each query point, search for neighbours only within the nearest few clusters rather than across the entire dataset.

Shared-memory centroid caching (reused). Both the K-Means partitioning phase (which reuses the `assign_clusters` kernel from Section 3) and the IVF query kernel cache all cluster centroids in shared memory. This amortizes global memory reads across all threads in a block, using the same technique described for K-Means above.

IVF index construction. Two lightweight kernels build the inverted index. The first (`ivf_count_clusters`) counts points per cluster using `atomicAdd`, and a CPU-side prefix sum produces an offset table. The second (`ivf_fill_lists`) scatters point indices into a flat `list_points[n]` array using atomic write cursors. The result is that all points belonging to cluster c are stored contiguously at `list_points[offsets[c] ...offsets[c+1]]`, enabling efficient sequential access during the query phase. All index buffers (counts, offsets, write pointers, and list points) are packed into a single `cudaMalloc`.

IVF query kernel. Each thread computes distances to all n_c centroids from shared memory, selects the n_p nearest clusters via repeated linear scan, and then searches only those clusters' point lists using the same max-heap PQ used in exact KNN. Histogram equalization is **fused** directly into the query kernel, just as in exact KNN.

Accuracy-first fallback for small inputs. When $n < 10{,}000$ or $k \leq 32$, the dispatcher bypasses

IVF entirely and runs the exact KNN kernel instead. As Table 1 shows, all three approaches take roughly 0.17–0.20 s for small n, since the runtime is dominated by CUDA context initialization and device memory allocation. The time penalty of running brute-force is therefore negligible, while guaranteeing zero MAE.

Adaptive parameter selection. For larger inputs, we set $n_c = n/k$ clusters and $n_p = \lceil k^2/n \rceil$ probes, clamped to sensible bounds. With these settings, the per-thread search cost works out to $O(k^2 \log k)$, which is **independent of** n. Increasing n_p improves recall at the cost of searching more candidates; in the limit, setting $n_p = n_c$ recovers exact KNN.

Complexity (query phase): $O(k^2 \log k)$ per thread, $O(nk^2 \log k)$ total; plus $O(Tn \cdot n_c)$ for the partitioning phase.

5 Runtime Analysis

We benchmarked all three approaches on a range of input sizes. The results are summarized in Table 1.

Table 1: Wall-clock runtime (seconds). GPU: NVIDIA GeForce RTX 2050 Laptop.

n	k	T	**Exact KNN**	**K-Means**	**Approx KNN**
1 000	8	10	0.273	0.207	0.178
5 000	16	10	0.172	0.172	0.170
10 000	32	20	0.199	0.176	0.201
20 000	64	20	0.433	0.183	0.237
50 000	64	30	1.002	0.196	0.484
50 000	128	30	1.825	0.200	0.672
100 000	64	30	2.041	0.226	1.558
100 000	128	50	3.968	0.241	2.458

Several patterns emerge from the data:

1. **Small inputs are dominated by overhead.** For $n \leq 10{,}000$, all three approaches take roughly 0.17–0.20 s regardless of mode. This is because CUDA context initialization and memory allocation dominate the actual computation time. This confirms that our strategy of falling back to exact KNN for small inputs in the approximate path is sound: there is no meaningful time penalty, but the accuracy is perfect.

2. **Exact KNN scales quadratically.** Going from $n = 10{,}000$ to $n = 100{,}000$ (a 10× increase) pushes the runtime from 0.20 s to 3.97 s (roughly 20×), which is consistent with $O(n^2)$ scaling modulo GPU parallelism gains at higher occupancy.

3. **K-Means stays nearly constant.** K-Means runtime ranges from just 0.17 s to 0.24 s across all input sizes. The $O(nk)$ per-iteration cost is well-absorbed by GPU parallelism, and early convergence keeps the actual number of iterations low.

4. **Approximate KNN offers a middle ground.** At $n = 100{,}000$, approximate KNN is about 1.6× faster than exact KNN (2.46 s vs. 3.97 s for $k = 128$), while maintaining low MAE thanks to spatial partitioning. The speedup grows with n because the IVF query cost is linear in n, unlike the quadratic cost of brute-force search.

5. **Sensitivity to k.** Doubling k from 64 to 128 at $n = 50{,}000$ nearly doubles KNN time (1.00 s → 1.83 s), since every thread must scan the same n points but with a larger heap. K-Means, by contrast, barely moves (0.20 s), because its cost is driven by the number of iterations rather than k itself.